

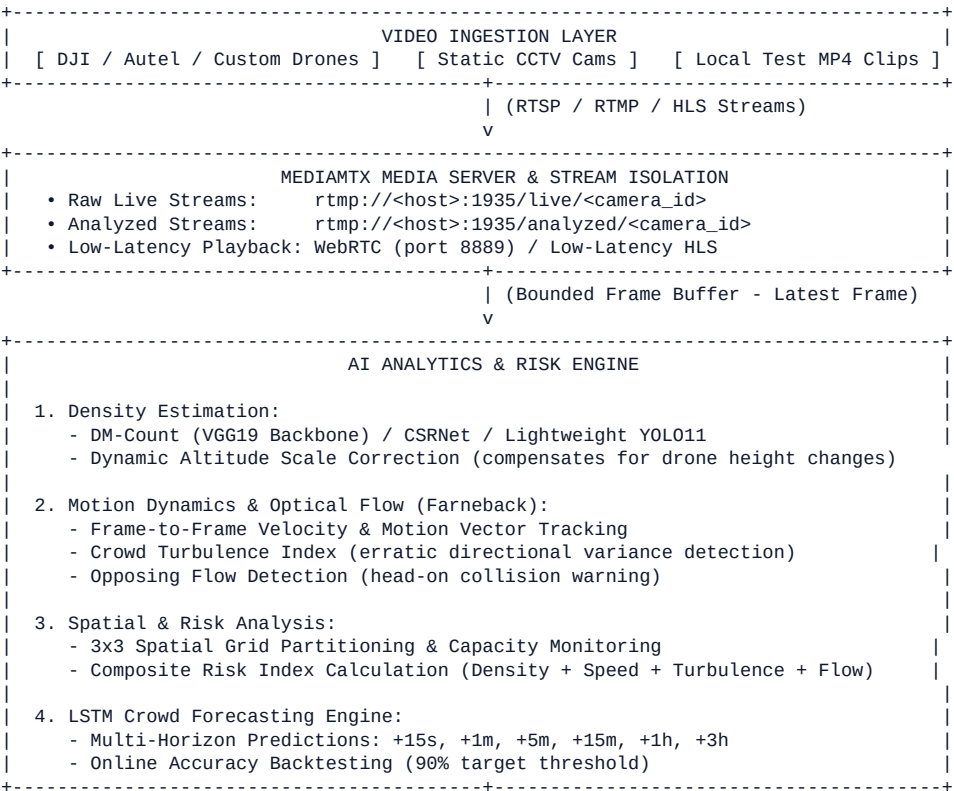
Pushkaralu Drone Crowd Monitor & Risk Engine — Comprehensive System Guide

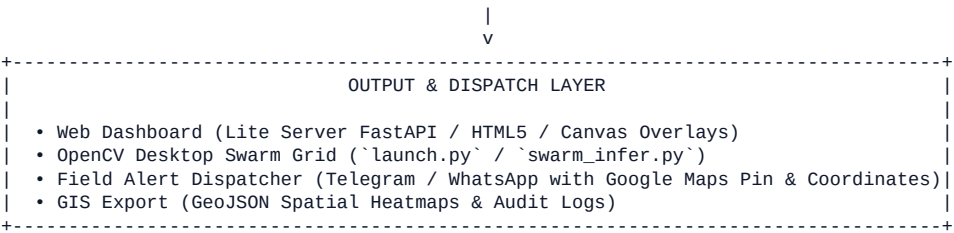
Executive Summary

The **Pushkaralu Drone Crowd Monitor & Risk Engine** is an advanced, AI-powered real-time video analytics platform built for high-density public event management, pilgrimage safety, and emergency response command centers.

By consuming raw video streams from drone swarms, static CCTV cameras, and mobile transmitters, the system dynamically estimates crowd counts, detects high-risk turbulence and counter-directional foot traffic, forecasts future crowd buildup using LSTM models, and dispatches automated geotagged alerts directly to security field teams.

1. System Architecture & Flowchart





2. Core Components & Deep Learning Stack

A. Deep Learning Density Models

- **DM-Count (Distribution Matching Count):** Primary deep learning model with a VGG19 backbone, trained on the NWPU crowd dataset. It models crowd density as continuous probability maps rather than discrete point detections, ensuring high accuracy in dense mass gatherings.
- **CSRNet (Dilated Convolutional Neural Network):** Secondary high-density crowd counter leveraging dilated convolutions to maintain spatial resolution without losing context.
- **YOLO11:** Lightweight bounding-box detector used for fast CPU/Edge inference in single-object detection or low-density environments.

B. Motion Dynamics Engine (Optical Flow)

- **Gunnar Farneback Dense Optical Flow:** Evaluates spatial movement vector fields between successive frames.
- **Speed Grid Tracking:** Divides the camera canvas into region-based speed fields to track crowd movement velocity ($\vec{v}$).
- **Turbulence Metric:** Quantifies chaotic vector directional variations:
$$\text{Turbulence} = \text{Var}(\theta_{\text{flow}}) \times ||\vec{v}||$$

High turbulence indicates bottlenecking, panic, or localized disturbance.
- **Opposing Flow Detection:** Identifies opposing velocity vectors within the same spatial grid to alert operators to dangerous head-on crowd collisions.

C. Altitude Scale Correction

When drones ascend, individuals appear smaller on the camera frame, leading to density underestimation. The engine applies an altitude scaling factor $S(h)$:

$$\text{Corrected Count} = \text{Raw Count} \times (h_{\text{current}} / h_{\text{baseline}})^{\gamma}$$

where h represents altitude and γ calibrates perspective compression.

D. Multi-Horizon LSTM Crowd Forecaster

- Evaluates count history sampled at 15-second intervals.
 - Generates crowd density forecasts across multiple horizons ($+15\text{ s}$, $+1\text{ m}$, $+5\text{ m}$, $+15\text{ m}$, $+1\text{ h}$, $+3\text{ h}$).
 - Employs a strict **90% accuracy gate**; models below target run in shadow mode and are continuously backtested against ground-truth frame history before deployment.
-

3. Dual Operational Modes

The system provides two execution modes depending on deployment needs:

Mode 1: Lite Web Dashboard (`run_lite.bat / lite_server.py`)

- **Target Use Case:** Command Center operators using browser-based web portals.
- **Architecture:** FastAPI backend + MediaMTX high-performance streaming server + HTML5 Canvas frontend.
- **Features:**
 - Live video grid supporting up to 40 simultaneous camera/drone feeds.
 - Zero-backlog latest-frame worker pipeline (clocked at 30 FPS).
 - WebRTC ultra-low-latency playback with automatic fallback to HLS.
 - One-click RTMP live feed ingestion for field mobile apps and drones.

Mode 2: High-Performance ML Swarm Command (`python launch.py / swarm_infer.py`)

- **Target Use Case:** Dedicated CUDA workstation with raw GPU inference.
 - **Architecture:** Python OpenCV multi-threaded engine + PyTorch GPU batch execution.
 - **Features:**
 - Multi-stream mosaic visual canvas with HUD risk indicators.
 - Real-time heatmaps, velocity vectors, and 3x3 spatial grid alerts.
 - Custom preset management for multi-drone stream configurations (`config.py`).
-

4. Risk Engine & Warning Thresholds

The system fuses density, motion, turbulence, and capacity into a unified **Composite Risk Score (0 - 100%)**:

$$\text{Risk Score} = w_d \cdot D_{\text{norm}} + w_v \cdot V_{\text{norm}} + w_t \cdot T_{\text{norm}} + w_c \cdot C_{\text{norm}}$$

Safety Level	Risk Score Range	Visual Indicator	Triggered Actions
NORMAL	<i>0% - 40%</i>	GREEN	Standard monitoring & routine telemetry log
MODERATE	<i>41% - 70%</i>	YELLOW	Highlight sub-zones; raise sample rate
WARNING	<i>71% - 85%</i>	ORANGE	Spatial grid flash alert; prepare dispatch
CRITICAL	<i>86% - 100%</i>	RED	Automated Telegram/WhatsApp field alert with Google Maps coordinates & stampede risk warning

5. Streaming & Ingestion Architecture

To prevent feedback loops and ensure smooth playback, the system uses strict stream separation:

1. Ingest Path (live/<camera_id>):

- Drones or camera apps stream uncompressed video directly to MediaMTX:

```
rtmp://<server-ip>:1935/live/drone-1
```

2. Analysis Pipeline:

- The inference thread reads frames from live/drone-1, runs DM-Count / YOLO / Optical Flow, draws HUD overlays, and writes to analyzed/drone-1.

3. Playback Path (analyzed/<camera_id>):

- Web dashboards consume processed streams via WebRTC (ws://<server-ip>:8889/analyzed/drone-1) or HLS.
- Separate ingestion and analyzed streams ensure the analyzer never consumes its own visual output.

6. Alert & Field Communications

When a sector crosses the **CRITICAL** threshold:

1. **Geotagged Dispatch:** The alert system formats GPS telemetry (*Lat, Lon*) into a field message:

■■ **STAMPEDE RISK ALERT — SECTOR 3 (Ghat 2)**

Density: 4.8 persons/m² (Capacity: 92%)

Turbulence: HIGH | Opposing Flow: DETECTED

■ **Location:** <https://maps.google.com/?q=16.9891,81.7823>

2. **GIS Export:** Writes live GeoJSON point and polygon layers to `outputs/geo_alerts.json` for integration into municipal GIS mapping software.

7. Operational Best Practices & Calibration

To ensure peak accuracy during live deployments:

A. Flight Operations & Drone Positioning

- **Nadir Camera Angle:** Angle camera $<60^\circ$ to $<90^\circ$ downward (top-down view). Avoid shallow diagonal shots to minimize perspective distortion.
- **Stable Hover:** Maintain stationary position during density sampling. Panning distorts optical flow vector calculation.
- **Altitude Sync:** Ensure configured software altitude matches drone altimeter.

B. Spatial Capacity Tuning

- Configure realistic square-meter capacities for sub-zones in `config.py` based on physical layout (e.g., narrow entry gates vs. open plaza squares).

C. Bandwidth & Network

- Use H.264 stream encoding at 30 FPS with 1280×720 resolution for transmission.
 - Ensure dedicated 5GHz Wi-Fi or LTE bonding routers between drones and the command station.
-

8. Command Reference & Setup

Quick Start Commands

```
# 1. Start Web Dashboard Portal (Lite Server + MediaMTX)
run_lite.bat

# 2. Interactive CLI Swarm Launcher
python launch.py

# 3. Single Stream Direct Inference
python infer.py --source rtmp://localhost:1935/live/drone-1 --gpu

# 4. Multi-Drone Swarm Command Mode
python swarm_infer.py

# 5. Train & Update LSTM Forecast Engine
python tools/extract_video_count_history.py
python tools/train_lstm_forecast.py outputs/crowd_history.csv outputs/video_count_history.csv
```

Key Configuration Files

- `config.py`: Primary configuration for thresholds, RTSP presets, GPU settings, and Telegram API credentials.
- `lite_server.py`: FastAPI web portal routing, camera indexing, and process execution manager.
- `SYSTEM_GUIDE.md`: System guide document.